Untying Prefill and Decode Loops in Looped Language Models

Yang Hu

Abstract

Looped language models reuse one block of layers several times per token and, under fixed-depth inference, run the same number of loops when they prefill the prompt and when they decode each answer token. We untie the two. A small *loop adapter* trained once on a frozen looped model lets it prefill with m loops and decode with n, for all 16 pairs up to four loops, from one stored prompt KV cache. Mapping accuracy over (m, n) on thirteen benchmarks shows that the best prefill count depends on the benchmark, its prompt and the decode budget. Within one adapter, prefilling 4-shot MATH-500 with one loop instead of four raises accuracy from 51.8 to 68.4 at four decode loops, while MMLU needs three or four prefill loops. Choosing m for each benchmark on held-out halves of its items beats the native Ouro-2.6B with the same number of decode loops significantly on 12 of 13 benchmarks with one decode loop (by 5 to 21 points) and on 10 of 13 with two; at three or four decode loops results are mixed. On Ouro-2.6B-Thinking, prefill 4 / decode 2 has higher point estimates than the native two-loop model on eight benchmarks (significantly on five), scores above a LoRA distillation of that model trained with the same data and teacher, and answers GSM8K in $0.55\times$ the time of the native four-loop model. Looped decision models show both directions: with more prefill than decision loops they beat a same-training control on 12 of 13 (m, n) pairs, and with fewer they fall below it on all 13. Code, results and per-item outputs: `https://github.com/yanghu819/untying-prefill-decode`.

1 Introduction

Looped language models such as Ouro (Zhu et al., 2025), Huginn (Geiping et al., 2025) and Mixture-of-Recursions (Bae et al., 2025) obtain depth without adding parameters by running a shared block of layers several times per token (Dehghani et al., 2018). Under the fixed-depth inference we study, the price is paid twice: every generated token runs every loop, and every loop keeps its own key–value (KV) cache.

Yet the two phases of inference are not alike. Prefill is parallel and paid once per request; decoding is sequential and paid for every token. A looped model nevertheless runs the same number of loops in both. What do prefill loops buy, and what do decode loops buy? If the answers differ, the two counts should be chosen separately.

We write prefill m / decode n for prefilling the prompt with m loops and decoding every answer token with n loops, and native n/n for the unmodified model, which uses n loops for both. When $m \neq n$, the decode loops must attend to prompt KV produced by a different loop than their own. Our *loop adapter* combines LoRA on the frozen model with a learned map on the prompt KV for every decode loop (Figure 2); one stored prompt cache serves every decode loop. We train one loop adapter and evaluate it at all 16 pairs (m, n) with its parameters fixed.

The resulting map (Figure 1) shows that the best prefill count depends on the benchmark, its prompt and the decode budget. Within one adapter, prefilling 4-shot MATH-500 with one loop instead of four raises accuracy from 51.8 to 68.4 at four decode loops; MMLU instead needs three or four prefill loops, and GSM8K and HumanEval are best at two and four. A native model can only move along the diagonal of this map. Choosing m for each benchmark on held-out halves of its items (Table 1) beats the native Ouro-2.6B with the same number of decode loops significantly on 12 of 13 benchmarks with one decode loop and on 10 of 13 with two; at three or four decode loops results are mixed, with 4-shot MATH-500 still 14 points ahead.

Untying is more than distillation. On Ouro-2.6B-Thinking, prefill 4 / decode 2 has higher

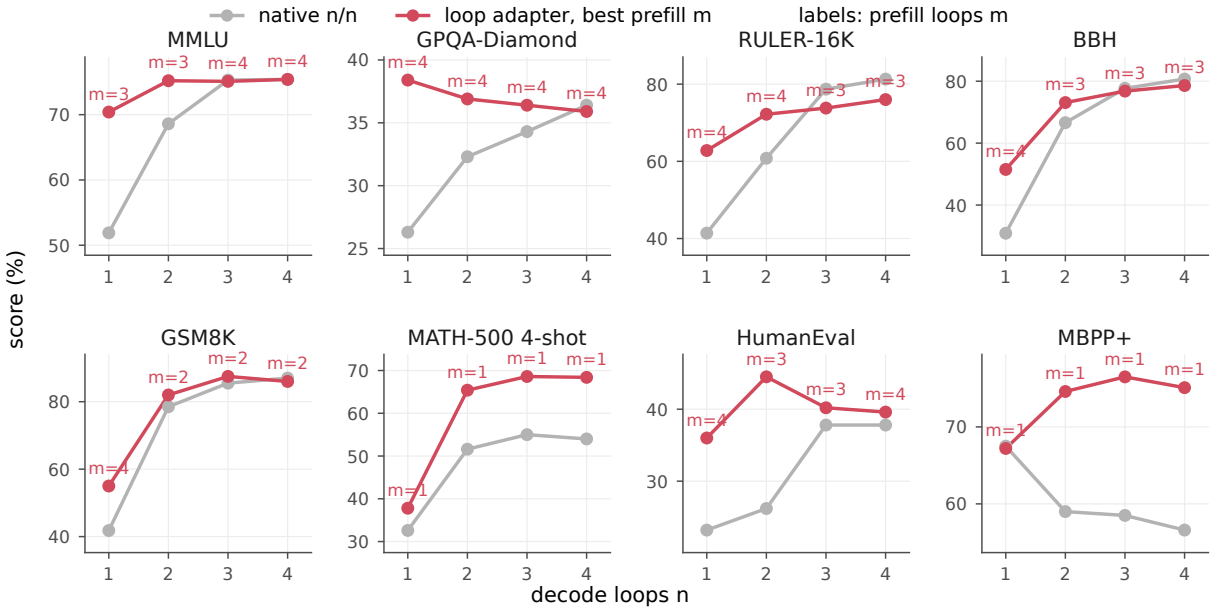


Figure 1: **The best prefill count depends on the task.** Ouro-2.6B, accuracy (%) against the number of decode loops n. Grey: the native model with n loops for prefill and decode (n/n). Red: one loop adapter with the best prefill count m at that n (labels). The best m differs by benchmark: one loop for 4-shot MATH-500 and MBPP+, three or four for MMLU and GPQA-Diamond. All thirteen benchmarks are in Figure 8; MMLU and GSM8K are subsets (20 questions per subject, 400 problems).

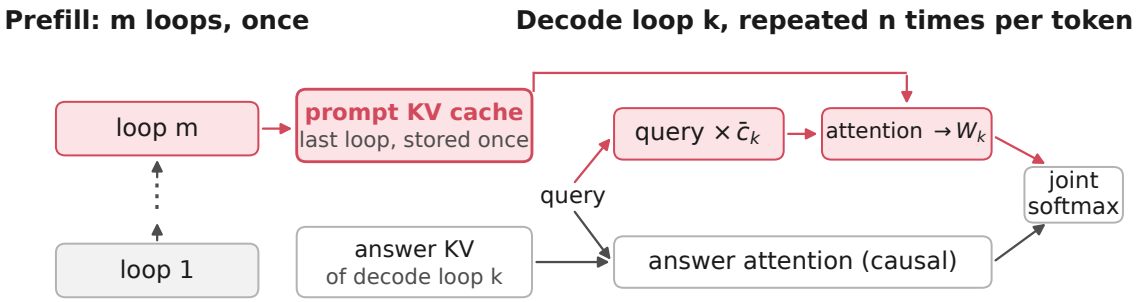


Figure 2: **The loop adapter.** The frozen model prefills the prompt with m loops and keeps one prompt KV cache. Each of the n decode loops reads that cache through its own map: a learned scale on each rotary (RoPE) pair of the keys, which can be moved onto the query so the cache is never rewritten, and a value map applied after attention. Attention over the prompt and over the generated tokens is computed separately and combined into one softmax (joint softmax), as in blockwise (FlashAttention-style) attention. The prompt KV is stored once for every n: total KV $b(\ell + n\,t)$ instead of $b\,n(\ell + t)$, with b bytes per token and loop and ℓ, t the prompt and answer length.

point estimates than native 2/2 on eight benchmarks (Figure 4) and above a LoRA distillation of native 2/2 trained with the same data and teacher, significantly on MMLU-Pro and ARC-Challenge; on 64 timed GSM8K problems it answers in $0.55\times$ the time of native 4/4 (Figure 6).

Untying also carries over to looped decision models, which score options over a long context without generating text (Section 4.5): prefilling the context with more loops than the decision uses beats a same-training control on 12 of 13 (m, n) pairs, and fewer prefill loops lose on all 13, by up to 13 points.

Our contributions are a measurement of what prefill loops and decode loops each buy in looped language models, a loop adapter that unties them on existing checkpoints with an exact single prompt cache, and an end-to-end measurement of the time it saves. Code, per-item results and a script that recomputes the paper's statistics accompany it.

2 Related Work

Looped and elastic depth. Accordion (Zhang et al., 2026) and Elastic Latent Recurrence (Ladhak et al., 2026) train one model for several depth budgets; ELR also finds that keeping only the last loop's KV retains most of the gain. The Parallel Loop Transformer (Wu et al., 2025a), LoopCoder-v2 (Yang et al., 2026) and shared-memory looped transformers (Monea et al., 2026) let later loops read the first loop's KV plus a short local window. All run the same number of loops for prefill and decode.

Phase-specific computation. Continuous Depth Batching (Schwethelm et al., 2026) exits early during decode; Position-Aware Depth Decay (Fan et al., 2025) skips more layers for later tokens; OverFill (Kim et al., 2025) decodes with a width-pruned model; Fixed Points Part II (Huang et al., 2026) distils a light prefill network; POP (He et al., 2026) and SPEED (Oh et al., 2026) prefill shallower than they decode; Decode-Branch Transformers (Liu et al., 2026), PHD (Wu et al., 2025b) and State-Prediction Separation (Langford et al., 2026) add decode-side computation. We instead vary prefill and decode loops independently within one loop adapter and measure what each buys.

Asymmetric KV producers and consumers. YOCO (Sun et al., 2024) has a self-decoder produce one global KV cache that a cross-decoder reads; YOCO-U (Sun et al., 2026) loops the self-decoder, keeping KV storage independent of the loop count while every token still runs the loops; KITE (Hu et al., 2026) expands a checkpoint into two towers so that bulk prefill runs only the KV-producing tower. These architectures are built for the split; we adapt an existing looped model after pretraining and choose the prefill loops m and decode loops n per request.

Sharing and transferring state. Looped Latent Attention (Neill and Reid, 2026) compresses KV across loops; BeNice! (Prairie et al., 2026) and LSTM-style looped transformers (Kirsch, 2026) share it; Cache-to-Cache (Fu et al., 2025), HeteroFold (Yun et al., 2026), Swarm Telepathy (Alabdulkareem et al., 2026) and CacheLink (Qiu et al., 2026) transfer KV between models, and BellowsKV (Zhang and Chen, 2026) shares one latent across layers. Like CacheLink, we train our maps through the receiver's output; within one model tokens, layers and positions align, so a RoPE-commuting scale and a value map suffice. For hybrid attention / linear-recurrence models (Yang et al., 2024; Dong et al., 2024), state-swap interventions (Afendulev et al., 2026) and LatentPort (Villani, 2026) study the two kinds of state; Appendix J relates them to prefill loops.

3 Method

We describe the loop adapter for Ouro; SenseNova-Loop-9B uses a lighter variant without maps (Appendix A).

Prefill and decode. "prefill m / decode n" prefills the prompt with m loops and decodes every answer token with n loops; native n/n is the unmodified model. The frozen model prefills the prompt with m loops and keeps only the last loop's prompt keys and values. The decoder is the same frozen model with rank-16 LoRA (Hu et al., 2021) on its attention projections, running n loops per token.

RoPE-commuting maps. Decode loop k reads the prompt KV cache through a map per (loop, layer, KV head). The key map is one complex scale c_j per rotary frequency pair j (Su et al., 2021): viewing the pair of key coordinates (k_{2j}, k_{2j+1}) as a 2-D vector, $c_j = r\,e^{i\phi}$ stretches it by r and turns it by ϕ. Writing a key pair rotated for position s as a complex number $\tilde{k}_j = e^{is\theta_j}k_j$, scaling commutes with the rotation, $c_j\tilde{k}_j = e^{is\theta_j}(c_j k_j)$, and $\mathrm{Re}\langle q_j, c_j\tilde{k}_j\rangle = \mathrm{Re}\langle \bar{c}_j q_j, \tilde{k}_j\rangle$: the map is applied to the query and the stored cache is never rewritten. The value map is a full linear map W_k per head; because attention is linear in the values, $\sum_i a_i W_k v_i = W_k \sum_i a_i v_i$ is applied once to the attention output over the prompt. Prompt and generated tokens share one softmax: the two parts are computed by two attention calls with outputs o_P, o_A and log-sum-exps λ_P, λ_A of their attention logits, and merged as $g\,o_P + (1-g)\,o_A$ with $g = \sigma(\lambda_P - \lambda_A)$, which equals one softmax over both (the blockwise softmax of FlashAttention (Dao, 2023)). The prompt KV is therefore stored once for every n, and total KV is $b(\ell + n\,t)$ instead of $b\,n(\ell + t)$. Serving from one cache equals materialising a mapped prompt cache per loop; in bfloat16 all 512 checked greedy tokens agree.

What the maps do. With the same training and the maps frozen at the identity, the decoder loses

```
# blocks: frozen looped block, L layers; m/n: prefill/decode loops
# c[k][l] = 1+a+1j*b per RoPE pair; W[k][l] = I+delta (init: I)
# LoRA on q, k, v, o is active only while decoding.

def prefill(prompt, m): # prompt minus its last token
    h = embed(prompt)
    for _ in range(m):
        for l in range(L):
            h, K[l], V[l] = blocks[l](h) # last loop's K, V kept
    return K, V # one prompt KV cache

def attend(q, K, V, kv_answer, c, W): # layer l, decode loop k
    qc = complex_mul(q, c.conj()) # key scale moved to the query
    o_p, lse_p = flash_attn(qc, K, V) # prompt part
    o_p = o_p @ W.T # attention is linear in V
    o_a, lse_a = flash_attn(q, *kv_answer, causal=True)
    g = torch.sigmoid(lse_p - lse_a) # exact softmax over both
    return g * o_p + (1 - g) * o_a

def decode(token, K, V, n): # last prompt token, then answer
    h = embed(token)
    for k in range(n):
        for l in range(L):
            h = blocks[l](h, attn=lambda q, kv: attend(
                    q, K[l], V[l], kv, c[k][l], W[k][l]))
    return lm_head(h)
```

```
# training: one (m, n) per micro-batch,
# teacher = the native 4/4 model
m, n = randint(1, 4), randint(1, 4)
with torch.no_grad():
    t = model(x, prefill=4, decode=4)
s = model(x, prefill=m, decode=n) # LoRA + maps
kl = kl_div(s.log_softmax(-1),
            t.log_softmax(-1), log_target=True,
            reduction="none").sum(-1)
loss = kl[answer].sum() / answer.sum()
```

Figure 3: **The loop adapter in PyTorch-style pseudocode.** The prompt KV cache is written once by the last of m prefill loops; every decode loop reads it through its own key scale (applied to the query) and value map (applied to the attention output), and the prompt and answer attention are combined exactly through their log-sum-exp.

little on short-context reasoning and knowledge but 11.4 points on RULER-16K at prefill 4 / decode 1 (Appendix H): the maps matter mainly for long prompts.

Training. We minimise $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ over answer positions, with the native 4/4 model as teacher, on 48M answer tokens of reasoning traces, instruction data, long-context data and web text. One (m, n) pair is sampled per micro-batch, so one set of parameters (63M, 2.4% of Ouro-2.6B) serves all 16 pairs. Details are in Appendix A.

4 Experiments

Setup. Models: Ouro-2.6B, Ouro-1.4B and their Thinking variants (Zhu et al., 2025) (four loops) and SenseNova-Loop-9B-Preview, a hybrid of Gated-DeltaNet and full-attention layers whose middle block is visited twice. Benchmarks cover knowledge (MMLU (Hendrycks et al., 2020), MMLU-Pro (Wang et al., 2024), ARC-Challenge (Clark et al., 2018), GPQA-Diamond (Rein et al., 2023)), long-context retrieval (RULER (Hsieh et al., 2024) at 4K/8K/16K, HELMET (Yen et al., 2024)), reasoning and instruction following (BBH (Suzgun et al., 2022), IFEval (Zhou et al., 2023)), math (GSM8K (Cobbe et al., 2021), MATH-500 (Hendrycks et al., 2021; Lightman et al., 2023)) and code (HumanEval (Chen et al., 2021), MBPP+ (Liu et al., 2023)); protocols and set sizes are in Appendix B. One loop adapter per model is trained once and evaluated at all 16 pairs. We compare with the native model at the same number of decode loops. Every comparison is paired over items with 95% bootstrap intervals.

4.1 The task map

Figure 1 shows accuracy at every (m, n). The best prefill count differs across benchmarks. The clearest case is 4-shot MATH-500: within the same adapter at four decode loops, one prefill loop scores 68.4 and four score 51.8 ($+16.6$ [12.4, 21.0]), and prefill 1 / decode 3 reaches 68.6 against 55.0 for native 3/3. MBPP+ is also best with one prefill loop. Knowledge goes the other way: MMLU rises from 52–53 with one prefill loop to about 70 with three or four at one decode loop and to 74–75 with more decode loops, and GPQA-Diamond is best at prefill 4 / decode 1. GSM8K is best with two prefill loops and HumanEval with four. Decode loops matter most for multi-step math: GSM8K rises from 46–55 at one decode loop to 71–88 at two or more. A native model can only move along the diagonal of this map.

4.2 Choosing the prefill loops per benchmark

For each benchmark and number of decode loops we choose m on half of its labeled items, score that choice on the other half, and swap (Table 1).

benchmark	decode 1	decode 2	decode 3	decode 4
MMLU	$+18.2^{*}$ [15.1, 21.4] (3,4)	$+5.8^{*}$ [3.8, 7.9] (3,4)	-0.3 [-1.5, 1.0] (3,4)	$+0.0$ [-1.1, 1.1] (4)
GSM8K	$+11.2^{*}$ [6.2, 16.2] (2,4)	$+1.5$ [-2.5, 5.5] (3,4)	$+2.0$ [-1.0, 5.0] (2)	-2.2 [-5.2, 0.8] (2,4)
MATH-500 4-shot	$+5.2^{*}$ [1.6, 8.8] (1)	$+13.8^{*}$ [9.6, 17.8] (1)	$+13.6^{*}$ [9.6, 17.8] (1)	$+14.4^{*}$ [10.2, 18.6] (1)
BBH 3-shot	$+20.9^{*}$ [18.0, 23.8] (4)	$+6.5^{*}$ [4.3, 8.8] (3)	-2.3^{*} [-4.1, -0.7] (4,2)	-1.7^{*} [-3.3, -0.1] (3)
ARC-C	$+14.8^{*}$ [12.2, 17.4] (4)	$+2.7^{*}$ [1.3, 4.2] (4,3)	$+0.9$ [-0.2, 1.9] (3)	-0.2 [-1.1, 0.7] (3)
MMLU-Pro	$+19.0^{*}$ [13.8, 24.3] (4,3)	$+5.7^{*}$ [1.2, 10.0] (4,3)	$+0.7$ [-3.6, 5.0] (4)	-2.1 [-5.7, 1.4] (4,3)
HumanEval	$+12.8^{*}$ [6.1, 20.1] (4)	$+18.3^{*}$ [11.6, 25.6] (3)	$+2.4$ [-1.8, 6.7] (3)	-0.6 [-5.5, 4.3] (3,4)
MBPP+	-0.3 [-3.4, 2.9] (1)	$+15.6^{*}$ [10.6, 20.9] (1)	$+18.0^{*}$ [12.7, 23.0] (1)	$+18.5^{*}$ [13.0, 23.8] (1)
GPQA-Diamond	$+12.1^{*}$ [5.1, 19.2] (4)	$+3.5$ [-3.0, 10.1] (3,4)	-1.0 [-7.6, 5.6] (4,2)	-3.5 [-8.6, 1.0] (3,4)
IFEval	$+9.4^{*}$ [5.5, 13.3] (4)	-0.9 [-4.4, 2.6] (4,3)	$+1.8$ [-1.1, 4.8] (2)	$+0.4$ [-2.8, 3.1] (4,2)
RULER-4K	$+12.3^{*}$ [8.5, 16.0] (2)	$+5.7^{*}$ [3.4, 8.0] (2)	-1.6 [-3.3, 0.0] (2)	-2.3^{*} [-4.1, -0.6] (2)
RULER-8K	$+17.6^{*}$ [13.4, 21.9] (3)	$+5.6^{*}$ [2.5, 8.7] (2,3)	-9.0^{*} [-11.6, -6.4] (2,3)	-10.3^{*} [-13.1, -7.7] (2,3)
RULER-16K	$+19.6^{*}$ [15.2, 23.6] (4,3)	$+10.4^{*}$ [6.4, 14.4] (4,3)	-5.6^{*} [-8.7, -2.6] (3,4)	-5.2^{*} [-8.0, -2.6] (3)

Table 1: **Choosing the prefill loops per benchmark, against the native model with the same decode loops.** Ouro-2.6B, one loop adapter. For each benchmark and number of decode loops n we choose the prefill count m on one half of the items, score it on the other half, and swap, so that no item is scored by a choice made on it; the table gives the resulting accuracy minus native n/n on the same items, the m chosen on each half (in parentheses), and paired 95% bootstrap intervals (*: interval excludes zero). Score points on a 0–100 scale.

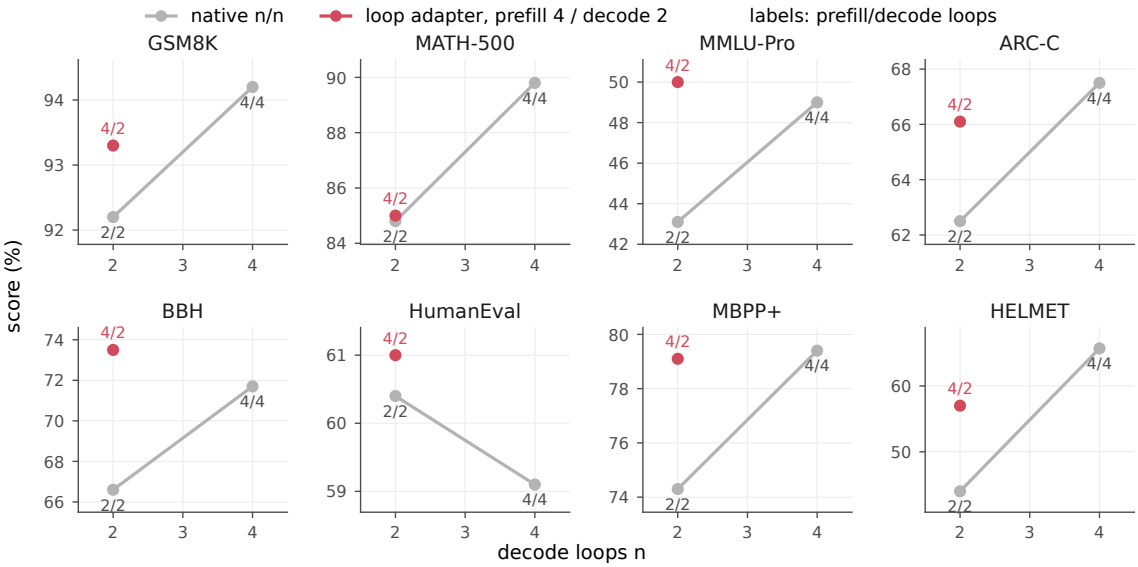


Figure 4: **Prefill 4 / decode 2: two decode loops, close to the quality of four.** Ouro-2.6B-Thinking, score against the number of decode loops. Grey: native 2/2 and 4/4. Red: the loop adapter with prefill 4 / decode 2. Same items for all three in each panel; HELMET is averaged over its categories. Table 5 gives every score, the paired intervals and the same adapter at prefill 2 / decode 2.

This calibrates m per benchmark; it is not routing per prompt, and it does not transfer to unseen tasks. Bootstrap intervals resample the held-out outcomes with the split and the chosen m fixed, so they do not include the variability of the choice. With one decode loop the chosen m beats native 1/1 significantly on 12 of 13 benchmarks, by 5 to 21 points, and ties on MBPP+, where native 1/1 is already strong; with two decode loops it wins significantly on 10 of 13, including HumanEval ($+18.3$) and MBPP+ ($+15.6$). With three or four decode loops results are mixed: the native model, which also prefills with three or four loops, is about as good on most benchmarks and better on long-context retrieval (RULER) and slightly on BBH, while 4-shot MATH-500 gains 13.6 and 14.4 points at three and four decode loops and MBPP+ 18.0 and 18.5. Ouro-1.4B shows the same pattern (5 of 5 and 3 of 5 benchmarks; Appendix E). Comparisons with native models also include the effect of training the adapter; comparisons within the adapter isolate the prefill loops. Equal decode loops do not mean equal latency (Section 4.4).

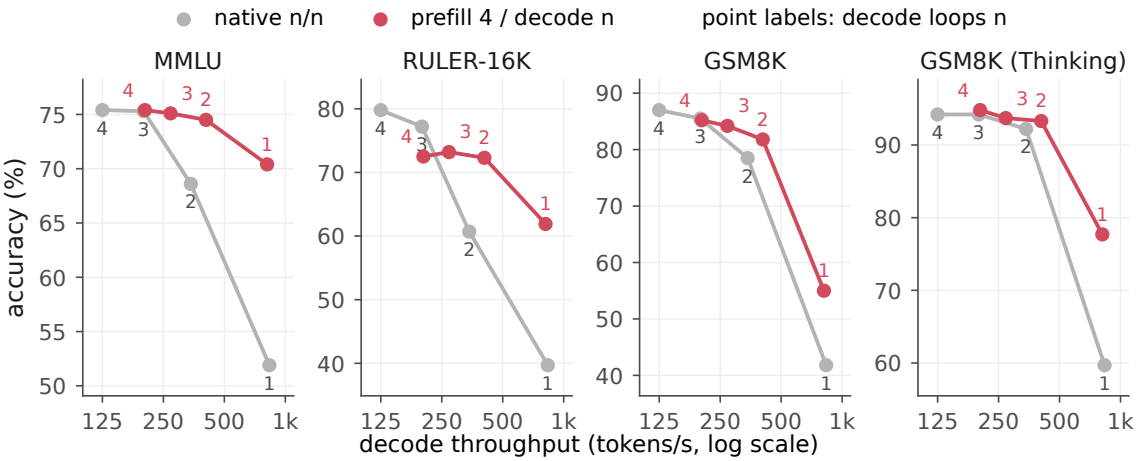


Figure 5: **Cutting decode loops costs less accuracy with the loop adapter.** Native n/n (grey) and the loop adapter with prefill 4 / decode n (red), $n = 1$–4 (labels). Throughput: best decode tokens/s over batch sizes at 8K context on one B300 (same for the base and Thinking models, which share the architecture); one stored prompt KV cache admits larger batches as n grows. Ouro-2.6B on the grid subsets of Figure 1; Ouro-2.6B-Thinking on all 1,319 GSM8K problems.

4.3 More prefill loops, fewer decode loops

Figure 4 puts the two findings together on Ouro-2.6B-Thinking: prefill 4 / decode 2. With the same number of decode loops as native 2/2, prefill 4 / decode 2 scores higher on all eight benchmarks, significantly on MMLU-Pro ($+6.9$ [3.1, 11.0]), BBH ($+6.9$ [4.6, 9.2]), ARC-Challenge ($+3.6$ [2.0, 5.3]), MBPP+ ($+4.8$ [1.3, 8.2]) and HELMET ($+13.0$ [6.7, 19.5]); on GSM8K, MATH-500 ($+0.2$ [-2.6, 3.0]) and HumanEval the gains are within noise. Against native 4/4, which decodes with twice the loops, it is within two points on six of the eight benchmarks and trails on HELMET (-8.7), a deficit concentrated in exact recall that this native-model contrast does not attribute to the prefill loops (Appendix K), and on MATH-500 (-4.8 [-7.8, -2.0]).

The same adapter at prefill 2 / decode 2 (Table 5) isolates the prefill loops: four instead of two add $+8.1$ [4.0, 12.1] on MMLU-Pro, $+7.5$ [1.4, 13.8] on HELMET, $+2.3$ [0.9, 3.7] on ARC-Challenge and $+2.2$ [0.1, 4.3] on BBH, with no detectable difference on GSM8K, MATH-500 and MBPP+; the same holds on Ouro-1.4B-Thinking (Appendix F).

Untying is not the same as distilling (Table 6). A LoRA on native 2/2, trained with the same data, teacher and number of tokens as the loop adapter, gains on HELMET and BBH but not on knowledge; prefill 4 / decode 2 scores above it on all eight benchmarks, significantly on MMLU-Pro ($+5.2$ [1.2, 9.3]) and ARC-Challenge ($+2.9$ [1.4, 4.4]). On this reasoning model four prefill loops beat one at two decode loops on all eight benchmarks, significantly on seven, which does not make four optimal (prefill 2 / decode 2 is slightly higher on MBPP+); unlike 4-shot MATH-500 on the base model (Section 4.1), one prefill loop does not help here.

Against measured decode throughput (Figure 5), prefilling with four loops makes the model more accurate at one or two decode loops and faster at three or four, because one stored prompt cache admits larger batches; prefill 4 / decode 2 runs at $1.19\times$ the throughput of native 2/2 and $3.2\times$ that of native 4/4.

4.4 Fewer decode loops are faster end to end

We time whole answers at batch 1, prefill included (Figure 6; Table 7 in Appendix G). On Ouro-2.6B-Thinking a loop adapter that decodes with two loops answers GSM8K in $0.55\times$ [0.45, 0.67] the time of native 4/4. On all 1,319 problems, prefill 4 / decode 2 scores 93.3 against 94.2, a paired difference of -1.0 [-2.0, 0.1]. Prefilling with 4 rather than 2 loops before decoding with two changes the answer time by $1.01\times$ [0.90, 1.11]: the saving comes from the adapted decoder with fewer loops, including its shorter answers. On MATH-500, whose answers it does not shorten, the saving is smaller: $0.63\times$ the time of native 4/4 but $1.22\times$ that of native 2/2 (Appendix G). On SenseNova-Loop-9B the trained prefill 1 / decode 1 scores 95.6 against 95.8 for native 2/2 on all 1,319 GSM8K problems and takes $0.59\times$ [0.52, 0.68] the time per an-

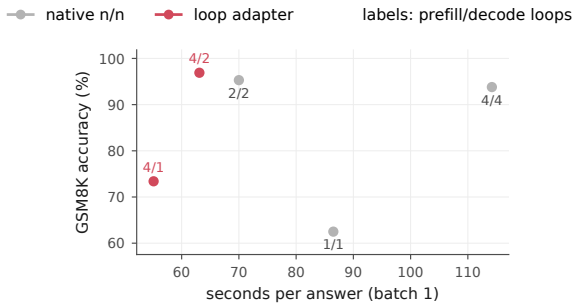


Figure 6: **Fewer decode loops save time end to end.** Ouro-2.6B-Thinking on GSM8K: accuracy against seconds per answer at batch 1 (prefill included, real greedy answers, the same 64 problems for every point; labels: prefill/decode loops). Prefill 4 / decode 2 takes $0.55\times$ the time of native 4/4; on all 1,319 problems it is 1.0 point below native 4/4 (Section 4.4). Native 1/1 is slower than 2/2 because its answers are longer. SenseNova-Loop-9B: Table 7.

swer, close to the ratio of layer executions per token (32 vs. 56).

4.5 Looped decision models: both directions

Untying is not specific to generation. Lev (Song et al., 2026) turns looped language models into decision models: they read a long context, then score the options of a question with a learned head after every loop, without generating text. We prefill the context with m loops, let each of the n decision loops attend to the last prefill loop's KV, and continue Lev's own LoRA and scoring head, once for every m; the control continues the same weights on the same data for the same number of updates with Lev's native n/n forward (Appendix I).

The sign of $m - n$ decides the outcome (Figure 7, Table 10). With more prefill than decision loops the model is significantly above the control on 12 of 13 (m, n) pairs, most with one decision loop, where prefilling with all of the backbone's loops gains $+5.3$ [3.4, 7.2] on Lev-1.4B, $+7.5$ [5.7, 9.5] on Lev-2.6B and $+4.4$ [2.8, 6.1] on Lev-3B. With fewer prefill than decision loops it is below the control on all 13 pairs (significantly on 10), by up to 13.0 [-14.7, -11.3] points (Lev-2.6B, prefill 1, four decision loops). On LevBench the context should be read with many loops and the decision can then stop early; on 4-shot MATH-500 the best prefill count was the smallest (Section 4.1).

5 Conclusion

A loop adapter makes the prefill and decode loops of an existing looped model independently configurable, with one stored prompt KV cache. The best prefill count depends on the checkpoint, the benchmark, its prompt and the decode budget; choosing it per benchmark pays most with one or two decode loops, where it beats native Ouro-2.6B on 12 and 10 of 13 benchmarks. On looped decision models the direction is clear-cut: more prefill than decision loops helps, fewer hurts.

Limitations

The prefill count is chosen per benchmark from half of its labeled items; we do not route individual prompts, and a choice need not transfer to an unseen task. The best count also depends on the prompt format: on the base model one prefill loop helps 4-shot MATH-500 but hurts zero-shot MATH-500 (Appendix K), and at two decode loops the Thinking model scores higher with four prefill loops than with one on all eight benchmarks we test (significantly on seven). With three or four decode loops the native model is better on long-context retrieval (RULER) and slightly on BBH. Gains against native models include training the adapter; the distillation control (Section 4.3) separates part of it. More prefill loops raise the time to first token, and equal decode loops do not mean equal latency; end-to-end time is measured at batch 1 on one GPU type, and throughput is a controlled decode benchmark. Thinking checkpoints on raw HELMET prompts can stop at once; we decode them so that an answer cannot be empty, for every configuration (Appendix B). All models are 1.4B–9B, decoding is greedy and benchmarks are English.

References

Kirill Afendulev, Alexey Dontsov, Elena Tutubalina, and Anton Korznikov. 2026. What Attention Recalls and Recurrence Controls in Hybrid Language Models. *arXiv preprint arXiv:2609.04434*.

Abdulrahman Alabdulkareem, Irene Tenison, Yang Zhang, Yada Zhu, Boris Katz, and Dan Gutfreund. 2026. Swarm Telepathy: Prefill-Free KV-Cache Transfer Across Families. *arXiv preprint arXiv:2610.15095*.

Sangmin Bae, Yujin Kim, Reza Bayat, Sungnyun Kim, Jiyoun Ha, Tal Schuster, Adam Fisch, Hrayr Harutyunyan, Ziwei Ji, Aaron Courville, and Se-Young Yun. 2025. Mixture-of-Recursions: Learning Dynamic Recursive Depths for Adaptive Token-Level Computation. *arXiv preprint arXiv:2507.10524*.

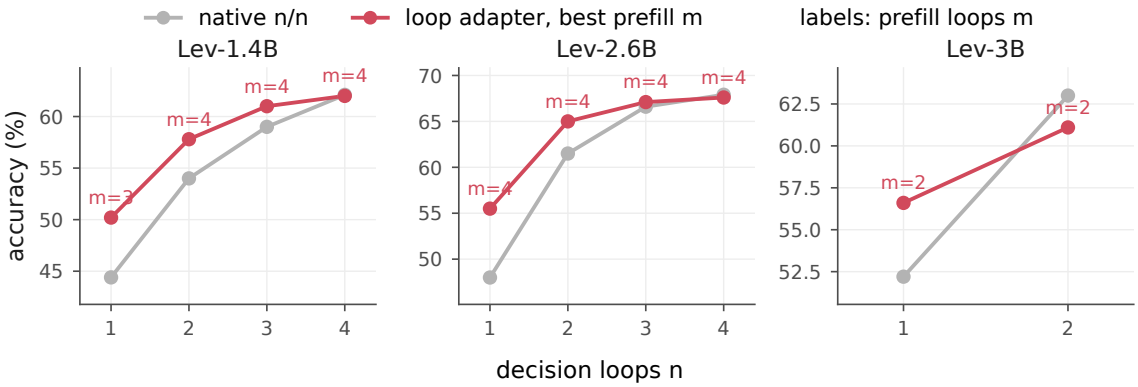


Figure 7: **Decision models gain when the context gets more loops than the decision.** Lev looped decision models (Song et al., 2026) on the public LevBench development set (3,270 answerable questions), accuracy against the number of decision loops n. Grey: native n/n, continued with the same data and updates as ours. Red: the best of the models trained to prefill the context with m loops (labels give m). Every (m, n) pair, both directions: Table 10.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, and 39 others. 2021. Evaluating Large Language Models Trained on Code. *arXiv preprint arXiv:2107.03374*.

Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. 2018. Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge. *arXiv preprint arXiv:1803.05457*.

Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training Verifiers to Solve Math Word Problems. *arXiv preprint arXiv:2110.14168*.

Tri Dao. 2023. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. *arXiv preprint arXiv:2307.08691*.

Mostafa Dehghani, Stephan Gouws, Oriol Vinyals, Jakob Uszkoreit, and Łukasz Kaiser. 2018. Universal Transformers. *arXiv preprint arXiv:1807.03819*.

Xin Dong, Yonggan Fu, Shizhe Diao, Wonmin Byeon, Zijia Chen, Ameya Sunil Mahabaleshwarkar, Shih-Yang Liu, Matthijs Van Keirsbilck, Min-Hung Chen, Yoshi Suhara, Yingyan Lin, Jan Kautz, and Pavlo Molchanov. 2024. Hymba: A Hybrid-head Architecture for Small Language Models. *arXiv preprint arXiv:2411.13676*.

Siqi Fan, Xuezhi Fang, Xingrun Xing, Peng Han, Shuo Shang, and Yequan Wang. 2025. Position-Aware Depth Decay Decoding (D^3): Boosting Large Language Model Inference Efficiency. *arXiv preprint arXiv:2503.08524*.

Tianyu Fu, Zihan Min, Hanling Zhang, Jichao Yan, Guohao Dai, Wanli Ouyang, and Yu Wang. 2025. Cache-to-Cache: Direct Semantic Communication Between Large Language Models. *arXiv preprint arXiv:2510.03215*.

Jonas Geiping, Sean McLeish, Neel Jain, John Kirchenbauer, Siddharth Singh, Brian R. Bartoldson, Bhavya Kailkhura, Abhinav Bhatele, and Tom Goldstein. 2025. Scaling up Test-Time Compute with Latent Reasoning: A Recurrent Depth Approach. *arXiv preprint arXiv:2502.05171*.

Junhui He, Zhihui Fu, Jun Wang, and Qingan Li. 2026. POP: Prefill-Only Pruning for Efficient Large Model Inference. *arXiv preprint arXiv:2602.03295*.

Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2020. Measuring Massive Multitask Language Understanding. *arXiv preprint arXiv:2009.03300*.

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring Mathematical Problem Solving With the MATH Dataset. *arXiv preprint arXiv:2103.03874*.

Cheng-Ping Hsieh, Simeng Sun, Samuel Kriman, Shantanu Acharya, Dima Rekesh, Fei Jia, Yang Zhang, and Boris Ginsburg. 2024. RULER: What's the Real Context Size of Your Long-Context Language Models? *arXiv preprint arXiv:2404.06654*.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. 2021. LoRA: Low-Rank Adaptation of Large Language Models. *arXiv preprint arXiv:2106.09685*.

Zhiheng Hu, Yixun Wei, Jian Zhou, Yizhuang Zhou, Ji Li, Xing Chen, Yang Li, Bojun Wang, Yibo Zhu,

Xiangyu Zhang, and Daxin Jiang. 2026. KITE: KV-Invariant Transformer Expansion for Efficient Agentic LLM Scaling. *arXiv preprint arXiv:2609.27294*.

Benhao Huang, Chufan Shi, Junlin Chen, Shicheng Wen, Zhengzhong Liu, Eric Xing, and Xuezhe Ma. 2026. Towards Looped Models Done Right, Part II: Rethinking at Fixed Points. *arXiv preprint arXiv:2610.06833*.

Woojeong Kim, Junxiong Wang, Jing Nathan Yan, Mohamed Abdelfattah, and Alexander M. Rush. 2025. OverFill: Two-Stage Models for Efficient Language Model Decoding. *arXiv preprint arXiv:2508.08446*.

Andreas Kirsch. 2026. LSTM-Style Looped Transformers with Shared KV Caches. *arXiv preprint arXiv:2610.33360*.

Faisal Ladhak, Shantanu Acharya, Santiago Akle Serano, Simeng Sun, Boris Ginsburg, and Krishna C Puvvada. 2026. One Model to Loop Them All: Elastic Latent Recurrence for Language Models. *arXiv preprint arXiv:2610.12582*.

John Langford, Nathan Godey, Giovanni Monea, Yoav Artzi, Harry Dong, Ying Fan, Gustavo de Rosa, and Zheng Zhan. 2026. Almost Free State Prediction Separation. *arXiv preprint arXiv:2609.03807*.

Hunter Lightman, Vineet Kosaraju, Yura Burda, Harri Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. 2023. Let's Verify Step by Step. *arXiv preprint arXiv:2305.20050*.

Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. 2023. Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation. *arXiv preprint arXiv:2305.01210*.

Liming Liu, Mingze Wang, and Tuo Zhao. 2026. Decode-Branch Transformers: Decoupling the Primary Prefill Path from Additional Decode Computation. *arXiv preprint arXiv:2608.12385*.

Giovanni Monea, Keshav Ramji, Yousef El-Kurdi, Luis A. Lastras, Yoav Artzi, Nathan Godey, and Ramón Fernandez Astudillo. 2026. The Surprising Effectiveness of Shared Memory in Looped Transformers. *arXiv preprint arXiv:2610.02383*.

James O' Neill and Fergal Reid. 2026. Looped Latent Attention: Cross-Loop KV Compression for Looped Transformers. *arXiv preprint arXiv:2607.15456*.

Jungsuk Oh, Hyeseo Jeon, Hyunjune Ji, Kyongmin Kong, and Jay-Yoon Lee. 2026. Shallow Prefill, Deep Decoding: Efficient Long-Context Inference via Layer-Asymmetric KV Visibility. *arXiv preprint arXiv:2605.06105*.

Hayden Prairie, Jacob M. Ryan, Taylor Berg-Kirkpatrick, and Daniel Y. Fu. 2026. BeNice! Closing Looping's Inference-Memory Gap via KV-Sharing. *arXiv preprint arXiv:2610.31633*.

Haomiao Qiu, Yining Tang, Zijia Song, Ruizi Han, Ziyue Qiao, and Miao Zhang. 2026. CacheLink: Functional KV Transport for Zero-Prefill Cross-Model Context Handoff. *arXiv preprint arXiv:2610.15509*.

David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. 2023. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv preprint arXiv:2311.12022*.

Kristian Schwethelm, Daniel Rueckert, and Georgios Kaissis. 2026. Depth-adaptive Inference of Looped Language Models via Continuous Depth Batching. *arXiv preprint arXiv:2608.09444*.

Dingjie Song, Zhiling Yan, Yuxuan Zhang, and Lichao Sun. 2026. LEV: Thinking Fast and Deep with Looped Decision Models. *arXiv preprint arXiv:2610.20951*.

Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, and Yunfeng Liu. 2021. RoFormer: Enhanced Transformer with Rotary Position Embedding. *arXiv preprint arXiv:2104.09864*.

Yutao Sun, Li Dong, Tianzhu Ye, Shaohan Huang, Jianyong Wang, and Furu Wei. 2026. Universal YOCO for Efficient Depth Scaling. *arXiv preprint arXiv:2604.01220*.

Yutao Sun, Li Dong, Yi Zhu, Shaohan Huang, Wenhui Wang, Shuming Ma, Quanlu Zhang, Jianyong Wang, and Furu Wei. 2024. You Only Cache Once: Decoder-Decoder Architectures for Language Models. *arXiv preprint arXiv:2405.05254*.

Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2022. Challenging BIG-Bench Tasks and Whether Chain-of-Thought Can Solve Them. *arXiv preprint arXiv:2210.09261*.

Simon P. Villani. 2026. LatentPort: Beyond KV Cache - Cross-Model Transfer of Recurrent Memory in Hybrid Language Models: A 4B-to-9B Hybrid-State Handoff Without Target Prefix Replay. *arXiv preprint arXiv:2609.25053*.

Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyan Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhu Chen. 2024. MMLU-Pro: A More Robust and Challenging Multi-Task Language Understanding Benchmark. *arXiv preprint arXiv:2406.01574*.

Bohong Wu, Mengzhao Chen, Xiang Luo, Shen Yan, Qifan Yu, Fan Xia, Tianqi Zhang, Hongrui Zhan, Zheng Zhong, Xun Zhou, Siyuan Qiao, and Xingyan Bin. 2025a. Parallel Loop Transformer for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2510.24824*.

Bohong Wu, Shen Yan, Sijun Zhang, Jianqiao Lu, Yutao Zeng, Ya Wang, and Xun Zhou. 2025b. Efficient Pretraining Length Scaling. *arXiv preprint arXiv:2504.14992*.

Jian Yang, Shawn Guo, Wei Zhang, Tianyu Zheng, Yaxin Du, Haau-Sing Li, Jiajun Wu, Yue Song, Yan Xing, Qingsong Cai, Zelong Huang, Chuan Hao, Ran Tao, Xianglong Liu, Wayne Xin Zhao, Mingjie Tang, Weifeng Lv, Ming Zhou, and Bryan Dai. 2026. LoopCoder-v2: Only Loop Once for Efficient Test-Time Computation Scaling. *arXiv preprint arXiv:2606.18023*.

Songlin Yang, Jan Kautz, and Ali Hatamizadeh. 2024. Gated Delta Networks: Improving Mamba2 with Delta Rule. *arXiv preprint arXiv:2412.06464*.

Howard Yen, Tianyu Gao, Minmin Hou, Ke Ding, Daniel Fleischer, Peter Izsak, Moshe Wasserblat, and Danqi Chen. 2024. HELMET: How to Evaluate Long-Context Language Models Effectively and Thoroughly. *arXiv preprint arXiv:2410.02694*.

Vincent-Daniel Yun, Woosang Lim, Haneul Yoo, Sungjoo Yoo, Murali Annavaram, and Sai Praneeth Karimireddy. 2026. Prefill-Free Cross-Family KV Cache Transfer for Heterogeneous Multi-Agent LLMs. *arXiv preprint arXiv:2609.32259*.

Jiamu Zhang and Hanjie Chen. 2026. BellowsKV: One Latent for Many Layers, Expanded on Demand. *arXiv preprint arXiv:2610.35023*.

Yudong Zhang, Wei Ding, Kaili Zheng, Yuchen Ding, and Zhenyu Zhang. 2026. Accordion: Elastic-Depth Inference via Fixed-Prefix Layer Looping. *arXiv preprint arXiv:2610.32492*.

Jeffrey Zhou, Tianjian Lu, Swaroop Mishra, Siddhartha Brahma, Sujoy Basu, Yi Luan, Denny Zhou, and Le Hou. 2023. Instruction-Following Evaluation for Large Language Models. *arXiv preprint arXiv:2311.07911*.

Rui-Jie Zhu, Zixuan Wang, Kai Hua, Tianyu Zhang, Ziniu Li, Haoran Que, Boyi Wei, Zixin Wen, Fan Yin, He Xing, Lu Li, Jiajun Shi, Kaijing Ma, Shanda Li, Taylor Kergan, Andrew Smith, Xingwei Qu, Mude Hui, Bohong Wu, and 14 others. 2025. Scaling Latent Reasoning via Looped Language Models. *arXiv preprint arXiv:2510.25741*.

A Training Details

Loop adapter. Rank-16 LoRA ($\alpha = 16$, no dropout) on the query, key, value and output projections, active only when decoding; per (decode loop, layer, KV head) a complex scale for each rotary frequency pair on keys and a full $D \times D$ value map, both initialised to the identity. Trainable parameters: 63,307,776 for Ouro-2.6B. The backbone is frozen.

Objective and data. $\mathrm{KL}(\text{teacher} \,\|\, \text{student})$ summed over answer positions and normalised by the answer tokens in the global batch; teacher = native 4/4 with the same weights; one pair (m, n) sampled per micro-batch. 48M answer tokens: OpenThoughts reasoning traces 45%, Tulu SFT responses 40%, LongAlpaca long-context answers 5% and FineWeb-Edu windows 10% (fractions of answer tokens); prompts up to 7,680 tokens, answers up to 1,024. Base models see a plain "User: … Assistant:" format; thinking models see the same samples in their chat template. No benchmark data was deliberately included; we did not decontaminate the training corpora.

Optimisation. AdamW ($\beta = (0.9, 0.95)$, no weight decay), peak learning rate 10^{-4} with 2% linear warm-up and cosine decay to 10^{-5}, gradient clipping at 1.0, global batch 24 sequences on 4 B300 GPUs (6.75 hours for Ouro-2.6B), bfloat16 backbone.

Shared answer KV cache. Generated tokens keep one KV cache per layer shared by all decode loops; at inference the last decode loop commits each token, and earlier loops read that history plus the current token's KV computed live in their own loop (*self-live*[1], a layout also used by LSTM-style looped transformers (Kirsch, 2026)). For example, when loop 1 processes token 5, it attends to tokens 1–4 through the last loop's stored KV and to token 5 through its own, freshly computed KV. Training uses the same KL objective in this layout, in two passes: a no-grad pass in which each loop reads its own causal answer KV records the last loop's answer KV; a second pass reads, at every loop, the prompt KV cache, that recorded history strictly before the current position, and the current token's KV computed live on the diagonal. Training thus approximates, but does not exactly reproduce, inference, where the history is generated recursively in the shared cache. With prompt and answer each stored once, total KV is $b(\ell + t)$, that of a one-loop model; with 1K-token prompts and 2,048 generated tokens the largest batch that fits grows from 32 (native 4/4) to 192 and decode-4 throughput from 219 to 476 tokens/s (759 vs. 586 at n=2), at a cost of 4.8 GSM8K points against native 4/4 (Table 9 compares it with per-loop answer caches). On Ouro-2.6B-Thinking, a loop adapter

[1] Self-live was developed by the author in separate work on shared KV caches for looped models; here it is applied to the loop adapter's answer tokens.

trained the same way changes full-set GSM8K at prefill 4 / decode 2 by $+0.5$ [-0.5, 1.6] against per-loop answer caches; at batch 1 its cost per generated token is the same (82.5 vs. 83.1 ms) but its answers are longer (1,166 vs. 741 tokens), so it takes $1.53\times$ [1.27, 1.84] the time per answer. Its benefit is memory and batch size, not batch-1 latency.

SenseNova-Loop-9B. Rank-16 LoRA on the attention projections of the looped layers (full attention: q/k/v/o; Gated-DeltaNet: input and output projections), active only when decoding; no maps. The one-visit decoder reads the prefill state of the last of $m \in \{1, 2\}$ visits (sampled 50/50), so the same decoder serves prefill 1 and prefill 2. The untrained condition passes the last visit's full-attention KV and Gated-DeltaNet convolution and recurrent states. Teacher: native 2/2; 20M answer tokens of the same mixture in the SenseNova chat template; peak learning rate 2×10^{-4}.

B Evaluation Details

Base models use lm-evaluation-harness prompts through our runner for GSM8K (3-shot CoT, flexible-extract, 1,319), MMLU (5-shot, 14,042), ARC-Challenge (25-shot, 1,172), BBH (3-shot CoT, 60 per subtask) and MATH-500 (the 500-problem subset of MATH, 4-shot Minerva prompt). RULER (13 tasks $\times$ 40 rows per length) and HELMET (RAG, synthetic recall, re-ranking and ICL categories; 1,108 rows) are prepared and scored by our own scripts. Thinking models answer GSM8K and MATH-500 zero-shot in their chat template, answers in `\boxed{}` scored with math-verify, with budgets of 4,096 and 8,192 tokens; their MMLU-Pro is 5-shot CoT with 30 questions per subject (420) and their BBH 3-shot CoT with 60 per subtask. SenseNova-Loop-9B uses MMLU-Pro with 70 questions per subject (980) through the harness and the full BBH (6,511) through our own scripts. For base models, MATH-500 answers are scored with math-verify on the full generated answer, with and without exemplars. The harness's own math-verify extraction gives -9.8 instead of -11.2 for the 4-shot comparison on Ouro-2.6B; Minerva's strict "Final Answer" match would reverse it (19.6 vs. 9.8) because the prefill-1 decoder drops the required format more often. On raw HELMET prompts thinking checkpoints can emit a newline and stop; for thinking models we therefore mask the stop tokens until an answer has a non-whitespace token, for every configuration. SenseNova RULER uses a non-thinking prefill for every configuration. Native throughput baselines: static per-loop caches with the same two-call flash attention (fastest), the standard dynamic cache (500 / 245 / 105 tokens/s at $n = 1/2/4$), and a preallocated contiguous cache with one attention call (267 / 130 / 57); all produce identical greedy tokens. Nanbeige4.2-3B-Base loop adapters trained per (m, n) pair, prefill 2 / decode 1 minus prefill 1 / decode 1: RULER 4K/8K/16K $+11.8/+11.7/+14.2$, MMLU $+12.2$, GSM8K $+5.2$. Intervals: paired bootstrap over items, 5,000 resamples; every interval in the paper is recomputed from released per-item scores by one script (Appendix L).

C Training-Seed Robustness

	task	seed 1	seed 2
prefill 4 $-$ prefill 1 (decode 1)	GSM8K	$+8.0$	$+7.2$
	MMLU	$+15.2$	$+15.2$
	RULER-4K	-3.8	-3.5
	RULER-8K	$+2.5$	$+2.0$
	RULER-16K	$+8.6$	$+8.6$
prefill 4 $-$ 2 (decode 2)	GSM8K	-1.7	-2.9
	RULER-4K	-7.2	-6.8

Table 2: Ouro-2.6B loop adapter trained with two seeds (initialisation and (m, n) sampling); paired differences within each seed, full sets.

D Full Task Map

E Other Checkpoints

Table 4 compares prefill counts at one decode loop within the same loop adapter on five checkpoints from two model families.

Table 3 repeats the per-benchmark choice on Ouro-1.4B. As on Ouro-2.6B, the gain is largest with one decode loop and shrinks with more; unlike Ouro-2.6B, one prefill loop gives no large gain on 4-shot MATH-500 or MBPP+, where the chosen count is mostly two or three.

F Prefill 4 / Decode 2 in Numbers, and on Ouro-1.4B-Thinking

Table 5 lists the scores and paired intervals behind Figure 4 and repeats the comparison on the smaller Thinking checkpoint. The effect of prefill loops replicates: with the same loop adapter, prefilling with four loops instead of two raises MMLU-Pro ($+8.6$), HELMET ($+7.3$), ARC-Challenge ($+2.7$)

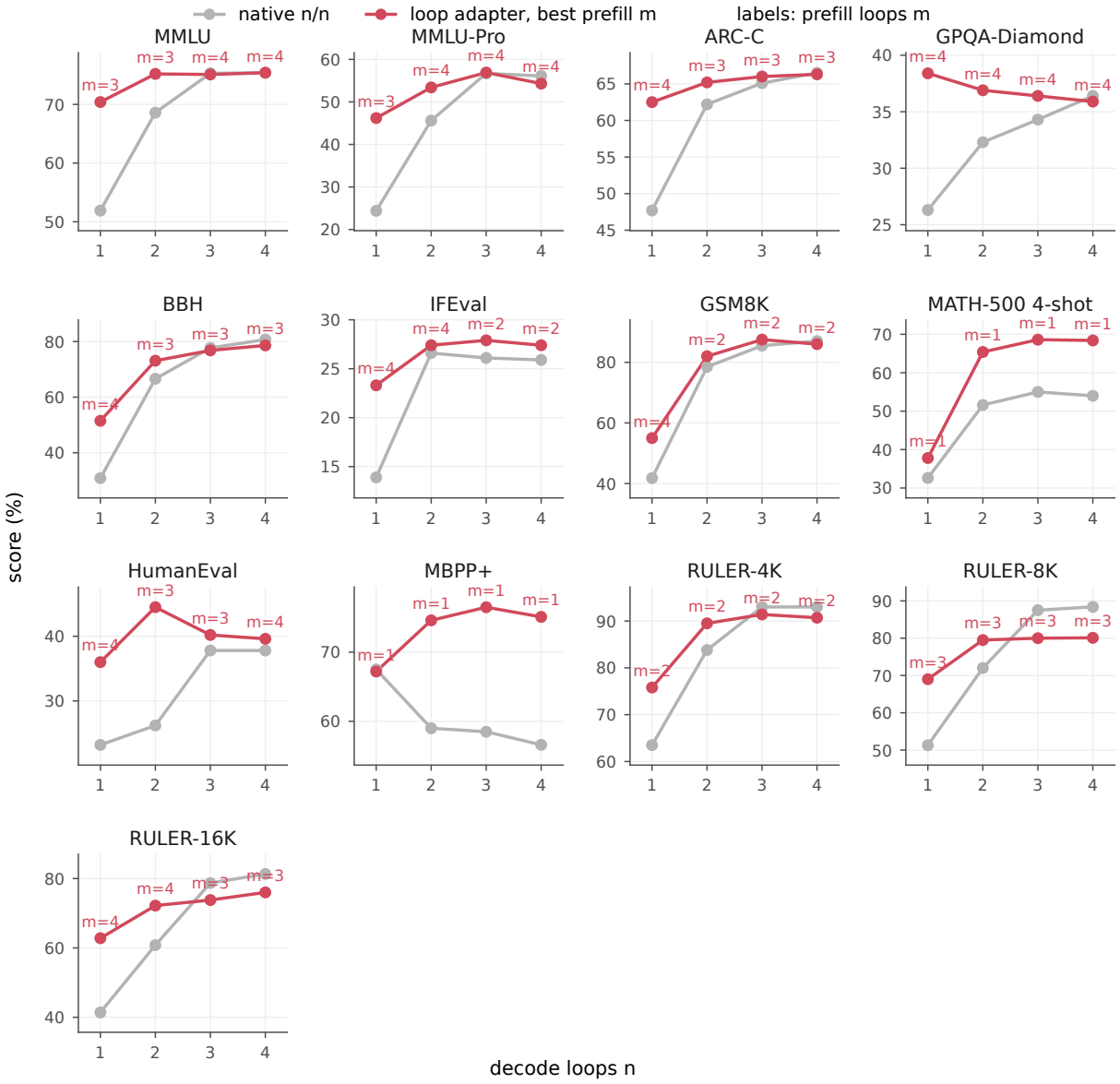


Figure 8: **All benchmarks of the task map** (Ouro-2.6B, one loop adapter; as Figure 1).

benchmark	decode 1	decode 2	decode 3	decode 4
MATH-500 4-shot	$+4.2^{*}$ [0.4, 8.0] (3)	$+0.6$ [-3.4, 4.4] (3)	-6.2^{*} [-10.0, -2.4] (3,2)	-2.0 [-5.4, 1.4] (1)
MBPP+	$+5.6^{*}$ [0.3, 10.8] (3)	-0.8 [-4.0, 2.4] (2)	-1.6 [-4.5, 1.3] (4,2)	-1.6 [-4.5, 1.3] (2)
HumanEval	$+8.5^{*}$ [3.0, 14.0] (1,4)	$+9.1^{*}$ [1.8, 16.5] (3)	$+2.4$ [-4.3, 9.1] (1,3)	$+4.3$ [-1.8, 10.4] (3)
MMLU-Pro	$+17.9^{*}$ [12.9, 23.1] (3)	$+6.2^{*}$ [1.7, 10.7] (3)	$+2.9$ [-1.0, 6.9] (3)	$+0.5$ [-3.1, 4.0] (4,3)
ARC-C	$+17.2^{*}$ [14.2, 20.1] (4)	$+4.2^{*}$ [2.1, 6.2] (3,4)	$+1.1$ [-0.4, 2.7] (4)	-0.1 [-1.3, 1.2] (4)

Table 3: **Table 1 on Ouro-1.4B** (its own loop adapter; five benchmarks): prefill loops chosen on one half of the items, scored on the other, minus native n/n on the same items.

and BBH ($+8.8$) and shows no detectable difference on GSM8K, MATH-500, HumanEval and MBPP+. Against native 2/2 the result is weaker than on Ouro-2.6B-Thinking: prefill 4 / decode 2 is significantly better on GSM8K ($+3.0$) and ARC-Challenge ($+4.3$), within noise on five benchmarks, and much worse on 3-shot BBH (-15.9), where this checkpoint's loop adapter degenerates at both prefill loop counts (Limitations). Against native 4/4 the gap is larger than on Ouro-2.6B-Thinking, most of all on BBH (-37.2) and HELMET (-18.3).

model (same decoder, decode 1)	knowledge	long context	multi-step
Ouro-2.6B, prefill 4 vs. 1	MMLU +15.2 [14.3, 16.1]	RULER-16K +8.6 [4.3, 12.8]	GSM8K +8.0 [5.3, 10.5]
Ouro-1.4B, prefill 4 vs. 1	ARC-C +10.8 [8.4, 13.1]	RULER-16K +1.4 [−2.6, 5.4]	GSM8K +6.2 [3.6, 8.7]
Ouro-2.6B-Thinking, prefill 4 vs. 1	MMLU-Pro +18.6 [13.6, 23.8]	HELMET +7.7 [2.1, 13.6]	GSM8K +6.6 [4.2, 9.2]
Ouro-1.4B-Thinking, prefill 4 vs. 1	MMLU-Pro +17.1 [12.4, 22.4]	HELMET +4.6 [0.1, 8.6]	GSM8K +13.6 [10.5, 16.6]
SenseNova-Loop-9B, prefill 2 vs. 1	MMLU-Pro +5.5 [2.9, 8.1]	HELMET +3.5 [0.8, 6.1]	BBH +7.0 [5.9, 8.0]

Table 4: **Prefill loops with the decoder held fixed** across five checkpoints (paired, within one loop adapter). Set sizes: MMLU 14,042, ARC-C 1,172, MMLU-Pro 420 (Ouro) and 980 (SenseNova), RULER 520, HELMET 1,108 (Ouro) and 6,646 (SenseNova), GSM8K 1,319, BBH 6,511. HELMET is averaged as its authors do (mean per dataset, then per category, then over its four categories). Each column shows one benchmark of that kind per checkpoint; counterexamples are in Appendix K. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

	score				adapter 4/2 minus		
benchmark	native 2/2	adapter 2/2	adapter 4/2	native 4/4	native 2/2	adapter 2/2	native 4/4
Ouro-2.6B-Thinking (Figure 4)							
GSM8K	92.2	93.0	93.3	94.2	+1.1 [−0.2, 2.4]	+0.2 [−1.0, 1.4]	−1.0 [−2.0, 0.1]
MATH-500	84.8	84.8	85.0	89.8	+0.2 [−2.6, 3.0]	+0.2 [−2.8, 3.2]	−4.8 [−7.8, −2.0]
MMLU-Pro	43.1	41.9	50.0	49.0	+6.9 [3.1, 11.0]	+8.1 [4.0, 12.1]	+1.0 [−2.6, 4.5]
ARC-C	62.5	63.8	66.1	67.5	+3.6 [2.0, 5.3]	+2.3 [0.9, 3.7]	−1.4 [−2.6, −0.1]
BBH	66.6	71.3	73.5	71.7	+6.9 [4.6, 9.2]	+2.2 [0.1, 4.3]	+1.9 [−0.4, 4.1]
HumanEval	60.4	55.5	61.0	59.1	+0.6 [−6.7, 7.9]	+5.5 [−0.6, 11.6]	+1.8 [−5.5, 8.5]
MBPP+	74.3	79.4	79.1	79.4	+4.8 [1.3, 8.2]	−0.3 [−3.7, 2.9]	−0.3 [−3.7, 2.9]
HELMET	44.0	49.5	57.0	65.7	+13.0 [6.7, 19.5]	+7.5 [1.4, 13.8]	−8.7 [−13.3, −4.1]
Ouro-1.4B-Thinking							
GSM8K	86.4	89.5	89.5	92.3	+3.0 [1.3, 4.8]	+0.0 [−1.5, 1.4]	−2.9 [−4.3, −1.5]
MATH-500	76.4	77.6	75.6	85.2	−0.8 [−4.6, 3.0]	−2.0 [−5.8, 1.8]	−9.6 [−12.8, −6.2]
MMLU-Pro	39.8	34.8	43.3	46.2	+3.6 [−1.0, 8.1]	+8.6 [4.3, 12.9]	−2.9 [−6.7, 1.2]
ARC-C	56.1	57.7	60.4	62.5	+4.3 [2.1, 6.4]	+2.7 [1.0, 4.5]	−2.0 [−3.7, −0.4]
BBH	48.7	24.0	32.8	70.0	−15.9 [−18.9, −13.0]	+8.8 [6.2, 11.5]	−37.2 [−39.9, −34.5]
HumanEval	34.1	33.5	34.1	32.9	+0.0 [−6.1, 6.1]	+0.6 [−5.5, 6.7]	+1.2 [−4.9, 7.3]
MBPP+	70.4	74.3	72.2	75.1	+1.9 [−2.1, 5.8]	−2.1 [−5.8, 1.9]	−2.9 [−6.3, 0.5]
HELMET	45.0	39.2	46.5	64.9	+1.5 [−4.8, 7.8]	+7.3 [2.2, 12.7]	−18.3 [−22.8, −13.6]

Table 5: **Figure 4 in numbers, and on Ouro-1.4B-Thinking.** Per checkpoint, one loop adapter serves prefill 2 / decode 2 and prefill 4 / decode 2; same benchmarks, protocols and items for both checkpoints. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

	score			distilled 2/2	prefill 4 / decode 2
benchmark	native 2/2	distilled 2/2	prefill 4 / decode 2	minus native 2/2	minus distilled 2/2
GSM8K	92.2	92.4	93.3	+0.2 [−1.0, 1.4]	+0.8 [−0.4, 2.0]
MATH-500	84.8	84.2	85.0	−0.6 [−3.8, 2.4]	+0.8 [−2.4, 3.8]
MMLU-Pro	43.1	44.8	50.0	+1.7 [−2.1, 5.2]	+5.2 [1.2, 9.3]
ARC-C	62.5	63.2	66.1	+0.7 [−0.4, 1.8]	+2.9 [1.4, 4.4]
BBH	66.6	71.3	73.5	+4.7 [2.7, 6.8]	+2.2 [0.0, 4.5]
HumanEval	60.4	55.5	61.0	−4.9 [−11.0, 1.2]	+5.5 [−1.8, 12.8]
MBPP+	74.3	75.9	79.1	+1.6 [−0.5, 3.7]	+3.2 [−0.5, 6.9]
HELMET	44.0	54.5	57.0	+10.5 [7.6, 13.7]	+2.5 [−3.6, 8.5]

Table 6: **Untying versus distillation** (Ouro-2.6B-Thinking). "Distilled 2/2": rank-16 LoRA on the native two-loop model, trained with the same data, teacher (native 4/4) and number of tokens as the loop adapter. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

G End-to-End Time and Decode Throughput

Serving. Storing the prompt once also changes throughput (Figure 5; Table 8 in Appendix G). In a controlled decode benchmark (8K context, 32 repeated decode steps, largest batch that fits on one B300), one prompt KV cache decodes $0.97\times$ as fast as the fastest native implementation we measured at one decode loop and 1.19–$1.62\times$ at two to four, because it admits larger batches as n grows;

task / model	config	acc. (%)	time (s)
GSM8K Ouro-2.6B-T	native 2/2	95.3	70.0
	native 4/4	93.8	114.2
	prefill 2 / decode 2	96.9	62.6
	prefill 4 / decode 2	96.9	63.1
MATH-500 Ouro-2.6B-T	native 2/2	85.0	243
	native 4/4	82.5	469
	prefill 4 / decode 2	85.0	298
GSM8K SenseNova	native 2/2	98.4	48.0
	prefill 1 / decode 1	96.9	28.4
	prefill 2 / decode 1	95.3	30.6

Table 7: **End to end per answer**, prefill included (batch 1, one B300; Ouro-2.6B-Thinking and SenseNova-Loop-9B; 64 GSM8K and 40 MATH-500 problems, budgets 4,096 and 8,192 tokens). On GSM8K, prefill 4 / decode 2 spends $1.27\times$ the decode time per generated token of native 2/2 at batch 1, but its answers are shorter.

this native baseline uses the same flash attention and is faster than the standard dynamic cache (Appendix B). With long answers (1K-token prompts, 2,048 generated tokens) one prompt KV cache decodes $0.97\times$ native at $n{=}2$ and $1.16\times$ at $n{=}4$.

8K context, 32 steps	$n{=}1$	$n{=}2$	$n{=}3$	$n{=}4$
native n/n	834	342	200	125
prefill 4 / decode d, one prompt cache	813	406	271	203
1K context, 2,048 steps		$n{=}2$		$n{=}4$
native n/n		586		219
prefill 4 / decode d, one prompt cache		570		253

Table 8: **Decode throughput** of Ouro-2.6B (tokens/s, best over tested batch sizes, one B300): native model against the loop adapter prefilling with 4 loops with one stored prompt KV cache.

H Design Variants

Table 9 compares design variants. Training the same loop adapter with its maps frozen at the identity, so that a LoRA decoder reads the last prefill loop's KV directly, gives small estimated losses on GSM8K and MMLU, with intervals including zero, but lowers RULER-16K by 11.4 points at prefill 4 / decode 1 and by 2.6–3.4 points at two decode loops. Restricting the key map to a RoPE-commuting scale, which is what allows one stored prompt KV cache, shows no measurable cost against unrestricted key maps with one cache per loop (RULER-16K, five (m, n) pairs).

Other design variants. Sharing generated-token KV across decode loops ("shared answer KV", Appendix A) shows no significant difference at two and three decode loops and costs 2.6 GSM8K points at four ($+0.3$ [-1.5, 2.1], -0.8 [-2.4, 0.8], -2.6 [-4.2, -1.1] at 4/2, 3/3, 4/4). A gated branch over the first prefill loop's KV improves RULER-4K by 2.3 [1.0, 3.6] points at prefill 4 / decode 1 and leaves GSM8K unchanged ($+0.7$ [-0.8, 2.2]).

variant	p/d	task	difference
maps frozen at identity[†]	4/1	RULER-16K	-11.4 [-14.7, -8.2]
	4/1	GSM8K	-1.7 [-3.6, 0.4]
	4/1	MMLU	-0.2 [-0.6, 0.1]
	4/2	RULER-16K	-2.6 [-4.5, -0.7]
	4/2	GSM8K	-0.2 [-1.5, 1.1]
	2/2	RULER-16K	-3.4 [-5.6, -1.3]
unrestricted keys[‡]	–	RULER-16K	all CIs include 0

Table 9: **Design variants** on Ouro-2.6B: variant minus our loop adapter at prefill m / decode n (full sets, paired). Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals. [†]Same data, budget, seed and (m, n) sampling; only the maps are not trained (LoRA only, the decoder reads the last prefill loop's KV directly). [‡]Unrestricted key maps with one prompt KV cache per decode loop, at prefill 1 / decode 1, 2/2, 3/1, 3/2 and 4/2.

I Looped Decision Models: Setup and Checks

We evaluate on the 3,270 answerable questions of LevBench's public development set (never its locked test set), with intervals over record groups as Lev does. Each prefill count m is one continued training (seed 0); the main comparison (m equal to the backbone's loops) was repeated with three more seeds and against a control trained for twice as many updates. With all decision loops, the model with the most prefill loops is no better than the control on the Ouro backbones and below it on Lev-3B (-1.8 [-3.0, -0.7]); with $m = n$ the trained models are within a few points of the control in both directions (Table 10).

Setup and checks. Both arms continue each released Lev model for 140 updates on the 8,925-record public S31 training subset with Lev's hyperparameters, from the same weights. Our reference forward runs the full sequence through both the prefill and the decision loops, so its training computation is about twice the control's. Without training, more prefill loops do not help (-3.0 on Lev-1.4B at one decision loop), and the control itself moves little against the released mod-

model	prefill m	decision 1	decision 2	decision 3	decision 4
Lev-1.4B	1	$+2.5^{*}$ [1.3, 3.7]	-2.8^{*} [−4.2, −1.4]	-6.1^{*} [−7.8, −4.6]	-9.6^{*} [−11.2, −7.9]
Lev-1.4B	2	$+2.9^{*}$ [1.3, 4.7]	$+2.6^{*}$ [1.4, 3.8]	-0.1 [−1.3, 1.1]	-2.7^{*} [−4.0, −1.4]
Lev-1.4B	3	$+5.7^{*}$ [3.9, 7.6]	$+3.8^{*}$ [2.3, 5.1]	$+1.8^{*}$ [0.7, 3.0]	-1.2 [−2.4, 0.0]
Lev-1.4B	4	$+5.3^{*}$ [3.4, 7.2]	$+3.8^{*}$ [2.3, 5.3]	$+2.0^{*}$ [0.7, 3.2]	-0.1 [−1.3, 1.1]
Lev-2.6B	1	$+0.7$ [−0.5, 2.0]	-7.5^{*} [−9.1, −5.9]	-12.5^{*} [−14.1, −10.8]	-13.0^{*} [−14.7, −11.3]
Lev-2.6B	2	$+4.4^{*}$ [2.6, 6.2]	-0.8 [−2.0, 0.4]	-3.5^{*} [−4.8, −2.3]	-4.1^{*} [−5.5, −2.8]
Lev-2.6B	3	$+6.8^{*}$ [4.8, 8.7]	$+2.3^{*}$ [0.9, 3.8]	-0.4 [−1.5, 0.7]	-0.9 [−2.0, 0.3]
Lev-2.6B	4	$+7.5^{*}$ [5.7, 9.5]	$+3.5^{*}$ [2.0, 5.0]	$+0.5$ [−0.7, 1.7]	-0.3 [−1.4, 0.8]
Lev-3B	1	$+0.3$ [−0.7, 1.3]	-8.1^{*} [−9.7, −6.6]	–	–
Lev-3B	2	$+4.4^{*}$ [2.8, 6.1]	-1.8^{*} [−3.0, −0.7]	–	–

Table 10: **Lev decision models, both directions.** Accuracy of the model trained to prefill the context with m loops minus the same-training control (native n/n) at n decision loops, LevBench development set (3,270 answerable questions), in points; brackets are 95% bootstrap intervals over record groups (*: interval excludes zero). Lev-3B's backbone has two loops. Above the diagonal ($m < n$) every difference is negative.

els at one loop ($+0.5$ on Lev-1.4B, $+1.3$ on Lev-2.6B). Repeating both arms with other seeds, the one-loop gain ranges from $+3.9$ to $+5.3$ on Lev-1.4B and from $+5.1$ to $+7.5$ on Lev-2.6B, every interval excluding zero (four seeds each); on Lev-3B it is $+4.4$, $+5.1$, $+4.0$ and $+0.9$ [−0.8, 2.7] (mean $+3.6$). A control trained for two epochs (twice the updates, about the training computation of the other arm) still trails it at one decision loop by $+4.0$ [2.1, 6.0] on Lev-1.4B, $+5.5$ [3.7, 7.4] on Lev-2.6B and $+4.0$ [2.3, 5.7] on Lev-3B.

J Prefill Loops in a Hybrid Model

SenseNova-Loop-9B stacks three Gated-DeltaNet layers per full-attention layer. Its prompt state therefore has two parts: the attention KV, which the decoder attends to, and each Gated-DeltaNet layer's convolution and recurrent state, a fixed-size summary of the prompt from which the decoder continues and which it keeps updating with its own tokens (the prefill stops before the last prompt token, which the decoder processes first). We give the one-visit decoder the second visit's state for one kind of layer and the first visit's for the other (Table 11). The loop adapter was trained only with both states from the first or both from the second visit, so we also retrain it, with the same recipe and seed, giving the decoder the second visit's state only in the Gated-DeltaNet layers for prefill-2 samples (below). These are experiments on one checkpoint; they show how the decoder responds to each state, not where information is stored.

Without training, either part alone recovers most of the gain of the full second-visit state (RULER-16K $+50.2$ and $+50.5$ against $+66.6$; MMLU-Pro $+23.7$ and $+21.5$ against $+36.2$). With the trained loop adapter the two parts behave differently. The second-visit Gated-DeltaNet state raises long-context scores: HELMET $+9.1$ [6.5, 11.6] and RULER-16K $+4.9$ [2.1, 7.7], against -0.2 and $+0.7$ for the second-visit attention KV, which lowers RULER-4K (-4.5 [−7.2, −2.0]). The second-visit attention KV raises MMLU-Pro ($+4.6$ [2.0, 7.2]; Gated-DeltaNet state alone $+2.0$ [−0.4, 4.5]). The ARC-Challenge loss of the full second-visit state (Table 12) appears with the Gated-DeltaNet part (-2.7 [−4.5, −0.9]; attention KV alone $+0.8$). In the "GDN state" condition the decoder still attends to the first visit's KV, consistent with exact retrieval relying on attention in hybrid models (Afendulev et al., 2026); what improves is the recurrent state it continues from. On Ouro, which has only attention layers, KV from more prefill loops likewise helps knowledge and loses exact recall (Appendix K). The same trained loop adapter, given the second-visit state only in its Gated-DeltaNet layers, scores above the full second-visit state on HELMET by 5.6 points and on RULER-4K/8K/16K by 6.5, 4.2 and 3.7, shows no detectable difference on ARC-Challenge, and scores below it on MMLU-Pro by 3.5. The loop adapter retrained on this mixed state confirms the pattern within its own training distribution: against its own prefill 1 / decode 1 it gains on HELMET ($+7.2$ [4.7, 9.7]) and RULER-16K ($+6.1$ [3.2, 9.1]) and loses ARC-Challenge (-2.8 [−4.7, −0.9]); against the original loop adapter at prefill 2 / decode 1 it is better on HELMET ($+4.4$ [2.7, 6.2]) and RULER-4K/8K/16K ($+6.7$, $+2.8$, $+2.9$) and worse on MMLU-Pro (-2.7 [−5.0, −0.3]). Which state to give the decoder is therefore a task-dependent trade-off that holds with and without training for it.

	untrained one-visit decoder			trained loop adapter		
benchmark	attention KV	GDN state	both	attention KV	GDN state	both
RULER-4K	+41.3 [37.2, 45.5]	+48.0 [44.0, 52.0]	+61.9 [57.8, 65.8]	−4.5 [−7.2, −2.0]	+2.7 [−0.5, 6.0]	−3.8 [−7.6, 0.2]
RULER-8K	+49.0 [44.8, 52.9]	+49.6 [45.7, 53.7]	+64.2 [60.0, 68.3]	−2.6 [−5.1, −0.2]	+2.3 [−0.8, 5.6]	−1.8 [−4.8, 1.3]
RULER-16K	+50.2 [46.0, 54.1]	+50.5 [46.4, 54.6]	+66.6 [62.7, 70.5]	+0.7 [−1.4, 2.8]	+4.9 [2.1, 7.7]	+1.2 [−1.7, 4.0]
MMLU-Pro	+23.7 [20.3, 27.0]	+21.5 [18.4, 24.8]	+36.2 [32.8, 39.8]	+4.6 [2.0, 7.2]	+2.0 [−0.4, 4.5]	+5.5 [2.9, 8.1]
ARC-C	+5.2 [3.2, 7.3]	+7.7 [4.9, 10.4]	+9.6 [6.7, 12.5]	+0.8 [−0.6, 2.1]	−2.7 [−4.5, −0.9]	−2.9 [−4.8, −1.0]
HELMET	+32.1 [30.1, 34.1]	+37.4 [35.5, 39.3]	+51.8 [49.8, 54.0]	−0.2 [−2.7, 2.2]	+9.1 [6.5, 11.6]	+3.5 [0.8, 6.1]

Table 11: **Which part of a hybrid model's prompt state benefits from a second prefill visit.** SenseNova-Loop-9B, one decode visit, after a two-visit prefill. Within the repeated block, "attention KV" gives the decoder the second visit's attention KV and the first visit's Gated-DeltaNet (GDN) convolution and recurrent states; "GDN state" the reverse; "both" the second visit's states throughout. Layers outside the block keep the two-visit prefill's states. Differences are to the one-visit prefill with the same decoder, in score points; brackets are paired 95% bootstrap intervals.

	score				prefill 2 / decode 1 minus	
benchmark	native 1/1	prefill 1 / decode 1	prefill 2 / decode 1	native 2/2	prefill 1 / decode 1	native 2/2
GSM8K	2.2	95.6	95.2	95.8	−0.4 [−1.4, 0.6]	−0.6 [−1.6, 0.4]
MATH-500	3.2	88.0	89.8	94.8	+1.8 [−1.0, 4.6]	−5.0 [−7.2, −2.8]
MMLU-Pro	18.9	60.3	65.8	68.2	+5.5 [2.9, 8.1]	−2.3 [−4.4, −0.2]
ARC-C	52.4	66.0	63.1	66.9	−2.9 [−4.8, −1.0]	−3.8 [−5.3, −2.4]
BBH	2.6	44.5	51.5	57.6	+7.0 [5.9, 8.0]	−6.1 [−7.0, −5.2]
HumanEval	3.0	75.0	76.2	81.7	+1.2 [−4.9, 7.3]	−5.5 [−11.0, 0.0]
HELMET	7.2	56.9	60.3	77.1	+3.5 [0.8, 6.1]	−16.8 [−18.7, −15.0]

Table 12: **SenseNova-Loop-9B at one decode visit.** One loop adapter serves prefill 1 / decode 1 and prefill 2 / decode 1; native 2/2 runs the repeated block twice per token (56 layer executions instead of 32). Same protocols as Table 4 in Appendix E (MMLU-Pro 980, HELMET 6,646, BBH 6,511 items). MBPP+ is omitted: its prompt forces the answer to start with code, and this thinking model then opens a reasoning block inside the code (native 1/1: 294, prefill 1 / decode 1: 236, prefill 2 / decode 1: 372, native 2/2: 375 of 378 answers), so its score measures this format failure. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

Table 12 is the SenseNova-Loop-9B counterpart of Figure 4, at one decode visit through the repeated block instead of two. With the same loop adapter, prefilling with two visits instead of one raises MMLU-Pro (+5.5), BBH (+7.0) and HELMET (+3.5), lowers ARC-Challenge (−2.9), and leaves GSM8K, MATH-500 and HumanEval within noise. Against native 2/2, prefill 2 / decode 1 shows no detectable difference on GSM8K (−0.6 [−1.6, 0.4]; the 0.59× answer time of Section 4.4 is measured for prefill 1 / decode 1) and scores lower on the other benchmarks, most on HELMET (−16.8); giving the decoder the second-visit state only in the Gated-DeltaNet layers narrows that HELMET gap to −11.2 [−13.0, −9.4]. The native one-visit model scores far lower on every benchmark.

K Prompt Format and HELMET Categories

HELMET shows which kinds of long-context task depend on the prefill loops (Table 13). On both Ouro reasoning models, prefilling with four loops instead of one improves retrieval-augmented QA by 15 points and re-ranking by 13–18, but lowers exact recall from a JSON key–value store on Ouro-1.4B-Thinking (−23.5); against native 4/4, prefill 4 / decode 2 gains on retrieval-augmented QA and in-context learning (+6.1, +6.6) and loses exact recall (58.8 vs. 100), so its HELMET average is lower (−8.7 [−13.3, −4.1]). SenseNova-Loop-9B does not follow the Ouro pattern on recall and re-ranking. The recall category is small (34 items on Ouro).

The effect of the prefill loops depends on the prompt format (Table 14). At one decode loop, more prefill loops give a negative but inconclusive estimate on RULER-4K, where answers are copied verbatim, and lower 4-shot MATH-500 on Ouro-2.6B; without exemplars the same comparison turns positive on two model sizes and two decode loop counts. Removing exemplars also changes length and format, so we read this as an interaction with the prompt.

comparison	RAG	re-rank	ICL	recall
2.6B-T, prefill $4-1$	$+15.4$	$+13.4$	$+5.1$	-2.9
1.4B-T, prefill $4-1$	$+14.5$	$+18.0$	$+9.3$	-23.5
SenseNova, prefill $2-1$	$+1.2$	-32.9	$+21.6$	$+24.0$
2.6B-T, 4/2 − native 4/4	$+6.1$	-6.3	$+6.6$	-41.2

Table 13: **HELMET by category** (score differences on a 0–100 scale; prefill comparisons at one decode loop, same loop adapter). More prefill loops help retrieval-augmented QA (RAG), re-ranking and in-context learning (ICL) on Ouro but can lose exact key–value recall. Items: 700 / 41 / 333 / 34 per category on Ouro, 4,200 / 246 / 2,000 / 200 on SenseNova; intervals are in `paper_numbers.json`.

model	decode n	task	prefill $4-$ prefill n
Ouro-2.6B	1	RULER-4K	-3.8 [-8.2, 0.3]
	1	MATH, 4-shot	-11.2 [-15.4, -6.8]
	1	MATH, 0-shot	$+7.0$ [3.0, 10.8]
	2	MATH, 0-shot	$+7.0$ [3.0, 11.2]
Ouro-1.4B	1	MATH, 4-shot	-4.0 [-8.2, 0.0]
	1	MATH, 0-shot	$+4.8$ [1.4, 8.4]

Table 14: **The effect of prefill loops depends on context length and prompt format.** Same loop adapter, base models, prefill 4 / decode n minus prefill n / decode n; MATH-500 scored with math-verify on the full answer. Differences are score points on a 0–100 scale; brackets are paired 95% bootstrap intervals.

L Reproducibility

`python ouro/scripts/paper_numbers.py compute` recomputes the paired differences, intervals, means, timing ratios and throughput of this paper from the accompanying per-item records (1 MB) in seconds on a laptop, and compares them with expected values hard-coded in the script, which mirror the numbers printed here. `REPRODUCE.md` maps every result to its scripts and gives the training and evaluation commands that regenerate the per-item records.